React Interview Prep

Topic 5 — useEffect: Side Effects, Lifecycle & Misconceptions

BEGINNER → INTERMEDIATE → EXPERT

■ Topic	useEffect — Side Effects, Cleanup & Lifecycle
■ Covers	3 forms · Cleanup · Stale closure · AbortController · 8 Misconceptions
■ Series	React Interview Prep — Session 5

by Akshay Chavhan • React Interview Series

1. What is useEffect?

A **side effect** is anything that reaches **outside** the component — fetching data, setting timers, subscribing to events, updating document.title, or manually touching the DOM. useEffect is React's dedicated place to run these.

> ■ ***useEffect runs AFTER the component renders and the browser has painted — not during render.***

2. The 3 Forms of useEffect

```
// Form 1 — runs after EVERY render (no array)
useEffect(() => { console.log('every render'); });

// Form 2 — runs ONLY on first render (empty array)
useEffect(() => { console.log('mount only'); }, []);

// Form 3 — runs on mount + when dependency changes
useEffect(() => { console.log('count changed'); }, [count]);
```

Form	Dependency Array	When It Runs
Form 1	No array	After every single render
Form 2	Empty [ ]	Only on mount (first render)
Form 3	[a, b]	On mount + when a or b changes

3. Cleanup Function

The function you **return** from useEffect is the cleanup. React runs it in two moments:

1. Before the next effect runs (dependency changed)
2. When the component unmounts (removed from DOM)

```
useEffect(() => {
const timer = setInterval(() => console.log('tick'), 1000);
return () => {
clearInterval(timer); // cleanup — stops the timer
};
}, []);
```

userId: 1 → 2

■ `effect runs for userId: 1` ■ `cleanup runs for userId: 1` ← `before next effect` ■ `effect runs for userId: 2`

4. AbortController — Fixing Race Conditions

When a dependency changes mid-flight (user clicks fast), the old fetch must be cancelled or stale data will overwrite fresh data.

```
useEffect(() => {
const controller = new AbortController();
fetch(`/api/users/${userId}`, { signal: controller.signal })
.then(res => res.json())
.then(data => setUser(data))
.catch(err => {
if (err.name === 'AbortError') return; // fetch was cancelled — ignore
console.error(err);
});
return () => controller.abort(); // cancel old fetch when userId changes
}, [userId]);
```

> ■ ***Without abort: if User 1 fetch is slower than User 2, User 1 data overwrites User 2 — wrong data shown.***

5. 8 Common Misconceptions About useEffect

Misconception 1 — useEffect runs during render

```
render runs
effect runs ← always AFTER browser paints, never during
```

> ■ *Truth: useEffect runs after the render is committed to DOM and browser has painted.*

Misconception 2 — Empty [] means 'run once forever'

■ **Wrong**

```
useEffect(() => {
console.log('once?');
}, []);
// Runs TWICE in React 18 Strict Mode dev!
```

■ **Correct**

```
// [] = 'no dependencies', not 'run once'
// Production: runs once ■
// Strict Mode dev: runs twice intentionally
// → exposes missing cleanup bugs
```

> ■ *Truth: [] means no dependencies. Strict Mode double-invokes effects in dev to catch cleanup bugs.*

Misconception 3 — useEffect can be async directly

■ **Wrong**

```
// ■ async returns a Promise
// Promise ≠ cleanup function → React warns
useEffect(async () => {
const data = await fetch('/api');
setUser(data);
}, []);
```

■ **Correct**

```
// ■ Define async inside, call immediately
useEffect(() => {
async function load() {
const res = await fetch('/api');
const data = await res.json();
setUser(data);
}
load();
}, []);
```

Misconception 4 — useEffect [] = componentDidMount exactly

componentDidMount runs **synchronously before** browser paints.

useEffect([]) runs **asynchronously after** browser paints.

■ Wrong

```
// May flicker — runs after paint
useEffect(() => {
const h = ref.current.offsetHeight;
setHeight(h);
}, []);
```

■ Correct

```
// No flicker — runs before paint
useLayoutEffect(() => {
const h = ref.current.offsetHeight;
setHeight(h);
}, []);
```

■ ***Truth: useLayoutEffect is the true componentDidMount equivalent for DOM measurements.***

Misconception 5 — Objects/Arrays in deps array are fine

■ Wrong

```
// New object reference every render!
const config = { theme: 'dark' };

useEffect(() => {
doSomething(config);
}, [config]); // ■ infinite loop!
```

■ Correct

```
// Fix 1: use primitive values
useEffect(() => {
doSomething(config);
}, [config.theme]); // ■ string

// Fix 2: useMemo to stabilize ref
const config = useMemo(
() => ({ theme: 'dark' }), []
);
```

■ ***Truth: React compares deps by reference. New object/array each render = always changed = infinite loop.***

Misconception 6 — Cleanup only runs on unmount

cleanup runs in TWO moments — not just unmount:

1. Before the next effect runs (when dependency changes)
2. When the component unmounts

```
// userId: 1 → 2
// ■ effect for userId: 1
// ■ cleanup for userId: 1 ← before next, NOT just unmount
// ■ effect for userId: 2
```

Misconception 7 — useEffect is the right place for all side effects

■ **Wrong**

```
// Extra render + unnecessary effect
useEffect(() => {
setDisplayName(userId.toUpperCase());
}, [userId]);

return {displayName};
```

■ **Correct**

```
// Just compute during render — no effect!
const displayName = userId.toUpperCase();

return {displayName};

// useEffect is for EXTERNAL world only
```

■ ***Truth: If you're transforming data you already have, do it in render. useEffect = external world only.***

Misconception 8 — Dependencies are optional, add what you want

■ **Wrong**

```
// Lying to React — name used but missing
useEffect(() => {
console.log(name, count);
}, [count]); // ■ name is stale forever!
```

■ **Correct**

```
// Honest deps — ESLint exhaustive-deps
useEffect(() => {
console.log(name, count);
}, [name, count]); // ■ always fresh
```

■ ***Truth: Deps array is a contract. If you use a value inside, it must be listed. Missing deps = stale closures.***

6. Misconceptions — Quick Reference

Misconception	Truth
Runs during render	Runs after browser paints
[] = run once forever	[] = no dependencies (twice in Strict Mode dev)
Can be async directly	Wrap async inside and call it
Same as componentDidMount	Use useLayoutEffect for DOM measurements
Object in deps is fine	New reference each render = infinite loop
Cleanup only on unmount	Also before next effect on dep change
Right place for all side effects	Only for external world — not data transforms
Deps are optional	Contract with React — missing = stale bugs

7. Interview Q&A;

Q **Beginner** **Q1. What is useEffect used for?**

➜ useEffect is for side effects — things that reach outside React like fetching data, setting up timers, subscribing to events, or updating document.title. It runs after the component renders so it never blocks the UI.

Q **Beginner** **Q2. What are the 3 forms of useEffect?**

➜ No dependency array → runs after every render. Empty array [] → runs only on mount. Array with values [a, b] → runs on mount and whenever a or b changes.

Q **Intermediate** **Q3. What is the cleanup function and when does it run?**

➜ It's a function returned from useEffect. React runs it before the next effect executes (when a dependency changes) AND when the component unmounts. Used to clear timers, cancel subscriptions, or abort fetch requests.

Q **Intermediate** **Q4. What is a stale closure in useEffect?**

➜ When an effect captures a variable at creation time and never updates — like reading count inside setInterval with an empty dep array. Fix: add the variable to dependencies, or use functional state updates that don't need to read the value.

Q Intermediate

Q5. Why can't useEffect be async directly?

➜ An async function returns a Promise. useEffect expects its return value to be either undefined or a cleanup function. A Promise is neither — React will warn you and the cleanup won't work correctly.

Q Expert

Q6. What is the difference between useEffect and useLayoutEffect?

➜ useEffect runs asynchronously after the browser paints — non-blocking. useLayoutEffect runs synchronously before the browser paints — like componentDidMount. Use useLayoutEffect only when reading DOM measurements to prevent visual flicker.

Q Expert

Q7. Why does useEffect run twice in development?

➜ React 18 Strict Mode intentionally mounts, unmounts, and remounts components to expose missing cleanup logic. If your effect breaks on double mount, your cleanup is incomplete. This only happens in development — not production.

Q Expert

Q8. How do you handle race conditions in useEffect?

➜ Use AbortController. Pass its signal to fetch, and call controller.abort() in the cleanup. When the dependency changes before the fetch completes, the old request is cancelled — preventing stale data from overwriting fresh data.